

DESARROLLO WEB BACKEND · INF560

UNIDAD 5

Despliegue a Producción

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

5.1 Introducción al despliegue de aplicaciones web

5.2 Alojamiento tradicional

5.2.1 Servidores dedicados

5.2.2 Servidores compartidos y VPS

5.3 Alojamiento en la nube

5.3.1 Principales plataformas: AWS, Google Cloud, Azure

5.3.2 Servicios PaaS e IaaS

5.4 Contenedores y virtualización

5.4.1 Introducción a Docker

5.4.2 Contenedores vs. máquinas virtuales

5.5 Arquitecturas escalables y de alta disponibilidad

5.5.1 Escalabilidad horizontal y vertical

5.5.2 Balanceo de carga

5.5.3 Tolerancia a fallos

COMPETENCIA DE LA UNIDAD

Comprende los modelos de alojamiento y despliegue de aplicaciones web —tradicional, en la nube y basado en contenedores— y analiza arquitecturas escalables y de alta disponibilidad, evaluando con criterio técnico la infraestructura más adecuada para llevar una aplicación backend desde el entorno de desarrollo hasta producción de forma segura, fiable y mantenible.

5.1 Introducción al despliegue de aplicaciones web

Durante el desarrollo, una aplicación backend se ejecuta en el computador del programador mediante un servidor local —por ejemplo `php artisan serve`— al que solo accede quien la construye. El **despliegue a producción** (o *deployment*) es el conjunto de actividades que llevan ese código desde el entorno de desarrollo hasta una infraestructura accesible de forma permanente y pública para los usuarios finales, garantizando rendimiento, disponibilidad y seguridad.

DEFINICIÓN · Despliegue (Deployment)

Proceso de instalar, configurar y poner en funcionamiento una aplicación en un entorno de ejecución —generalmente un servidor accesible desde Internet— de modo que quede disponible para los usuarios reales con las garantías de rendimiento y seguridad propias de un sistema en operación.

Una práctica fundamental es separar el ciclo de vida del software en **entornos** diferenciados, cada uno con su propia configuración y propósito. Esta separación evita que cambios no probados afecten a los usuarios y permite validar el sistema de manera progresiva.

Entorno	Propósito	Características
Desarrollo (development)	Programar y probar cambios localmente.	Datos ficticios, depuración activada, recarga en caliente.
Pruebas / Staging	Réplica de producción para validación previa.	Configuración idéntica a producción, datos de prueba.
Producción (production)	Atender a los usuarios reales.	Máximo rendimiento, errores ocultos, monitoreo y respaldos.

El stack de ejecución en el servidor

Una aplicación backend rara vez se ejecuta sola. En producción interviene un **stack** de componentes que cooperan para atender cada petición: un **servidor web** (Nginx o Apache) recibe las peticiones HTTP y sirve los archivos estáticos; un **intérprete del lenguaje** (PHP-FPM, Node.js, etc.) ejecuta la lógica de la aplicación; y un **sistema gestor de base de datos** (PostgreSQL, MySQL) persiste la información. A esto suelen sumarse servicios de **caché** (Redis) y de **colas** para tareas en segundo plano.

EJEMPLO · Pasos típicos de despliegue de una aplicación Laravel

El despliegue de un proyecto Laravel a producción sigue una secuencia reconocible que conviene automatizar.

```
# 1. Obtener la última versión del código
git pull origin main

# 2. Instalar dependencias optimizadas (sin paquetes de desarrollo)
composer install --no-dev --optimize-autoloader

# 3. Aplicar migraciones de base de datos
php artisan migrate --force
```

```
# 4. Cachear configuración, rutas y vistas para mayor rendimiento
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Secuencia básica de despliegue. En la práctica se ejecuta mediante un script o una herramienta de CI/CD.

ADVERTENCIA · Las credenciales nunca van en el código

La configuración sensible —contraseñas de base de datos, claves de API, la `APP_KEY`— se gestiona mediante **variables de entorno** (archivo `.env`), que jamás debe subirse al repositorio. En producción, además, `APP_DEBUG` debe estar en `false` para no exponer trazas de error con información interna del sistema.

5.2 Alojamiento tradicional

El **alojamiento tradicional** (o *hosting* clásico) agrupa los modelos en los que la aplicación se ejecuta sobre servidores físicos concretos, generalmente alquilados a un proveedor que opera un centro de datos. El cliente contrata una capacidad fija —de un servidor completo o de una porción de él— y asume, en mayor o menor grado, su administración. Es un modelo anterior a la nube, pero todavía muy vigente por su simplicidad y costo predecible.

5.2.1 Servidores dedicados

Un **servidor dedicado** es una máquina física completa puesta al servicio exclusivo de un único cliente. Toda la capacidad de cómputo —procesador, memoria, disco y red— está reservada para sus aplicaciones, sin compartirla con terceros. El cliente obtiene **acceso de administrador** (`root`) y control total sobre el sistema operativo, el software instalado y la configuración de seguridad.

Ventajas	Desventajas
Rendimiento máximo y predecible: ningún otro cliente compite por los recursos.	Costo elevado, tanto de alquiler como de operación.
Control absoluto sobre el sistema y la seguridad.	El cliente es responsable del mantenimiento, parches y actualizaciones.
Aislamiento total frente a otros usuarios.	Escalar exige migrar a hardware más potente (proceso manual y lento).

NOTA · ¿Cuándo elegir un servidor dedicado?

Resulta apropiado para aplicaciones con cargas altas y sostenidas, requisitos estrictos de cumplimiento normativo, o necesidades muy específicas de hardware. Para proyectos pequeños o de tráfico variable suele resultar sobredimensionado y poco eficiente en costos.

5.2.2 Servidores compartidos y VPS

En el **alojamiento compartido** (*shared hosting*), un mismo servidor físico aloja decenas o cientos de sitios web pertenecientes a clientes distintos, que comparten procesador, memoria y disco. Es la opción más económica y sencilla, pero ofrece poco control y rendimiento limitado: un pico de tráfico en un sitio vecino puede degradar el desempeño de los demás (el llamado problema del *vecino ruidoso*).

El **servidor privado virtual (VPS, Virtual Private Server)** representa un punto intermedio. Mediante **virtualización**, un servidor físico se divide en varias máquinas virtuales independientes, cada una con recursos garantizados, su propio sistema operativo y acceso `root`. El cliente disfruta de buena parte del control de un servidor dedicado a una fracción de su costo, aunque el hardware subyacente siga siendo compartido.

Criterio	Compartido	VPS	Dedicado
Costo	Muy bajo	Medio	Alto
Control / acceso root	No	Sí	Sí
Recursos garantizados	No	Sí	Sí (todos)
Aislamiento	Bajo	Medio-alto	Total
Ideal para	Sitios pequeños	Apps en crecimiento	Cargas altas

5.3 Alojamiento en la nube

El **cómputo en la nube (cloud computing)** es un modelo en el que los recursos informáticos —servidores, almacenamiento, bases de datos, redes— se ofrecen como un servicio a través de Internet, de forma **bajo demanda** y con **pago por uso**. En lugar de comprar y mantener hardware propio, la organización consume capacidad de proveedores que operan grandes centros de datos, y paga únicamente por lo que utiliza.

DEFINICIÓN · Características esenciales de la nube (NIST)

El cómputo en la nube se define por cinco rasgos: autoservicio bajo demanda (el usuario aprovisiona recursos sin intervención humana), amplio acceso por red, agrupación de recursos compartidos entre múltiples clientes, elasticidad rápida (la capacidad crece o decrece según la carga) y servicio medido (se factura con precisión lo consumido).

Frente al modelo tradicional, la nube aporta **elasticidad** —la infraestructura se adapta automáticamente a picos y valles de tráfico— y convierte una gran inversión inicial en hardware (CAPEX) en un gasto operativo variable (OPEX). A cambio, introduce dependencia del proveedor y exige nuevas competencias para gestionar costos y configuraciones de forma eficiente.

5.3.1 Principales plataformas: AWS, Google Cloud, Azure

Tres proveedores concentran la mayor parte del mercado mundial de nube pública. Aunque sus catálogos son equivalentes en lo esencial, cada uno tiene su propia terminología y fortalezas.

Plataforma	Cómputo	Bases de datos	Notas
Amazon Web Services (AWS)	EC2, Lambda	RDS, DynamoDB	El proveedor más extenso y maduro; mayor cuota de mercado.
Google Cloud (GCP)	Compute Engine, Cloud Run	Cloud SQL, Firestore	Fortaleza en datos, contenedores y aprendizaje automático.

Plataforma	Cómputo	Bases de datos	Notas
Microsoft Azure	Virtual Machines, Functions	Azure SQL, Cosmos DB	Fuerte integración con el ecosistema empresarial Microsoft.

NOTA · Almacenamiento de objetos

Un servicio transversal a las tres plataformas es el **almacenamiento de objetos** —Amazon S3, Google Cloud Storage, Azure Blob Storage—, ampliamente usado para guardar archivos subidos por los usuarios (imágenes, documentos) fuera del servidor de la aplicación. Laravel, por ejemplo, integra S3 de forma nativa a través de su sistema de discos del componente `Storage`.

5.3.2 Servicios PaaS e IaaS

Los servicios en la nube se clasifican según **cuánta responsabilidad asume el proveedor** frente al cliente. Cuanto más alto el nivel, menos infraestructura debe gestionar el equipo de desarrollo, a cambio de menor control sobre los detalles.

Modelo	Qué gestiona el proveedor	Qué gestiona el cliente	Ejemplos
IaaS (Infraestructura)	Hardware, virtualización, red.	Sistema operativo, runtime, aplicación, datos.	EC2, Compute Engine, VMs de Azure.
PaaS (Plataforma)	Todo lo anterior + SO y runtime.	Solo la aplicación y sus datos.	Heroku, App Engine, Laravel Cloud.
SaaS (Software)	Toda la pila, hasta la aplicación.	Únicamente el uso y la configuración.	Gmail, Dropbox, Salesforce.

En **IaaS** el cliente alquila máquinas virtuales "vacías" y conserva el control que tendría con un VPS, pero con la elasticidad de la nube. En **PaaS** el desarrollador solo entrega su código y la plataforma se encarga de ejecutarlo, escalarlo y mantener el sistema operativo: ideal para acelerar el despliegue sin administrar servidores. Existe además el modelo **serverless o FaaS** (*Functions as a Service*, como AWS Lambda), donde se ejecutan funciones individuales que se activan ante eventos y se facturan por milisegundos de ejecución.

EJEMPLO · Compromiso control vs. comodidad

Desplegar la misma API en una instancia EC2 (IaaS) implica instalar y mantener Nginx, PHP y PostgreSQL manualmente, con máximo control. Hacerlo en una PaaS como Laravel Cloud reduce el trabajo a conectar el repositorio y pulsar "desplegar", a costa de adaptarse a las convenciones de la plataforma.

5.4 Contenedores y virtualización

Un reto recurrente del despliegue es el clásico "*en mi máquina funciona*": diferencias entre el entorno de desarrollo y el de producción —versiones de PHP, extensiones, librerías del sistema— que provocan fallos difíciles de reproducir. La **virtualización** y, sobre todo, los **contenedores** surgen para empaquetar una aplicación junto con todo lo que necesita y garantizar que se ejecute igual en cualquier entorno.

5.4.1 Introducción a Docker

Docker es la plataforma de contenedores más extendida. Permite empaquetar una aplicación y sus dependencias en una unidad estandarizada y portable llamada **imagen**, que luego se ejecuta como un **contenedor**: un proceso aislado del resto del sistema pero que comparte el núcleo del sistema operativo anfitrión.

Concepto	Descripción
Dockerfile	Receta de texto con las instrucciones para construir una imagen.
Imagen (image)	Plantilla inmutable que contiene la aplicación y sus dependencias.
Contenedor (container)	Instancia en ejecución de una imagen; efímera y aislada.
Registro (registry)	Repositorio de imágenes, como Docker Hub, del que se descargan o publican.

EJEMPLO · Dockerfile para una aplicación PHP/Laravel

Una imagen base de PHP sobre la que se copia el código y se instalan las dependencias.

```
# Imagen base oficial con PHP 8.3 y el servidor FPM
FROM php:8.3-fpm

# Instalar extensiones necesarias para Laravel y PostgreSQL
RUN apt-get update && apt-get install -y libpq-dev \
    && docker-php-ext-install pdo pdo_pgsql

# Directorio de trabajo dentro del contenedor
WORKDIR /var/www

# Copiar el código de la aplicación
COPY . .

# Instalar dependencias de Composer optimizadas para producción
RUN composer install --no-dev --optimize-autoloader

# Puerto que expone el contenedor
EXPOSE 9000

CMD ["php-fpm"]
```

Dockerfile de ejemplo. Cada instrucción genera una capa reutilizable en la imagen.

NOTA · Comandos esenciales de Docker

Construir una imagen: `docker build -t mi-app .` — Ejecutar un contenedor: `docker run -p 80:9000 mi-app` — Listar contenedores activos: `docker ps`. Para orquestar varios servicios juntos (aplicación + base de datos + caché) se emplea Docker Compose mediante un archivo `docker-compose.yml`.

5.4.2 Contenedores vs. máquinas virtuales

Contenedores y máquinas virtuales resuelven el mismo problema —aislar y empaquetar aplicaciones— pero con arquitecturas muy distintas. Una **máquina virtual** (VM) virtualiza el hardware completo: sobre un *hipervisor* se ejecuta un **sistema operativo invitado entero** por cada VM. Un **contenedor**, en cambio, virtualiza solo a nivel del sistema operativo: todos los contenedores

comparten el núcleo del anfitrión y empaquetan únicamente la aplicación y sus dependencias, sin un sistema operativo propio.

Criterio	Máquina virtual	Contenedor
Aislamiento	Sistema operativo completo por VM.	Comparten el núcleo del anfitrión.
Tamaño	Gigabytes.	Megabytes.
Tiempo de arranque	Minutos.	Segundos o menos.
Consumo de recursos	Alto (cada VM duplica el SO).	Ligero y eficiente.
Grado de aislamiento	Más fuerte (hardware virtual).	Menor (nivel de proceso).

ADVERTENCIA · No siempre es uno u otro

Contenedores y VMs no son excluyentes. En la práctica es habitual ejecutar contenedores Docker dentro de máquinas virtuales en la nube, combinando el aislamiento fuerte de la VM con la ligereza y portabilidad de los contenedores.

5.5 Arquitecturas escalables y de alta disponibilidad

A medida que una aplicación gana usuarios, un único servidor deja de ser suficiente y aparece el riesgo de que su caída deje el servicio inaccesible. Dos propiedades guían el diseño de la infraestructura para afrontarlo: la **escalabilidad** —capacidad de atender más carga— y la **alta disponibilidad** —capacidad de seguir operando pese a los fallos—.

5.5.1 Escalabilidad horizontal y vertical

Existen dos estrategias fundamentales para aumentar la capacidad de un sistema. La **escalabilidad vertical** (*scale up*) consiste en dar más recursos a una sola máquina: más procesador, memoria o disco. La **escalabilidad horizontal** (*scale out*) consiste en añadir más máquinas que trabajen en paralelo y se reparten la carga.

Criterio	Vertical (scale up)	Horizontal (scale out)
Cómo crece	Una máquina más potente.	Más máquinas iguales.
Límite	El hardware máximo disponible.	Prácticamente ilimitado.
Complejidad	Baja: no cambia la arquitectura.	Mayor: requiere coordinación.
Punto único de fallo	Sí (una sola máquina).	No (carga distribuida).
Costo	Crece de forma exponencial.	Crece de forma lineal.

ADVERTENCIA · Aplicaciones sin estado (stateless)

Para escalar horizontalmente, la aplicación debe ser **sin estado**: ninguna petición puede depender de datos guardados en la memoria de un servidor concreto, pues la siguiente petición del mismo usuario podría atenderla otro servidor. Por eso las sesiones, la caché y los archivos se externalizan a servicios compartidos (Redis, base de datos, almacenamiento de objetos) accesibles por todas las instancias.

5.5.2 Balanceo de carga

Cuando varias instancias atienden la aplicación, hace falta un componente que reparta las peticiones entrantes entre ellas: el **balanceador de carga** (*load balancer*). Actúa como punto de entrada único: recibe todo el tráfico y lo distribuye según un algoritmo, evitando que un servidor se sobrecargue mientras otros quedan ociosos.

Algoritmo	Funcionamiento
Round Robin	Reparte las peticiones por turnos, una a cada servidor en orden cíclico.
Least Connections	Envía cada petición al servidor con menos conexiones activas.
IP Hash	Asigna cada cliente siempre al mismo servidor según su dirección IP.
Weighted (ponderado)	Da más peticiones a los servidores más potentes según un peso asignado.

El balanceador también ejecuta **comprobaciones de salud** (*health checks*): consulta periódicamente a cada servidor y, si uno deja de responder, lo retira automáticamente de la rotación para no enviarle tráfico. Según el nivel en que operan, se distingue el balanceo de **capa 4** (transporte, por IP y puerto) del de **capa 7** (aplicación, capaz de decidir según el contenido HTTP).

EJEMPLO · Balanceo con Nginx

Nginx puede actuar como balanceador definiendo un grupo de servidores y reenviándoles el tráfico.

```
upstream backend {  
    least_conn;                # algoritmo: menos conexiones  
    server 10.0.0.1:9000;  
    server 10.0.0.2:9000;  
    server 10.0.0.3:9000;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://backend;  
    }  
}
```

Configuración de Nginx que distribuye las peticiones entre tres instancias del backend.

5.5.3 Tolerancia a fallos

La **tolerancia a fallos** es la capacidad de un sistema para seguir funcionando, total o parcialmente, aun cuando alguno de sus componentes falla. Es el fundamento de la **alta disponibilidad** (*high availability*), que se mide por el porcentaje de tiempo en que el servicio permanece operativo.

DEFINICIÓN · Punto único de fallo (SPOF)

Un Single Point of Failure es cualquier componente cuya caída detiene todo el sistema. Eliminar los SPOF —mediante redundancia— es el objetivo central del diseño de alta disponibilidad: ningún elemento crítico debe existir en una sola copia.

Las técnicas principales para lograr tolerancia a fallos son la **redundancia** (duplicar componentes críticos: varios servidores, balanceadores, copias de la base de datos), la **conmutación por error**

o *failover* (que un componente de reserva asuma el trabajo automáticamente cuando el principal cae) y la **replicación** de datos junto con **respaldos** periódicos.

La disponibilidad se expresa habitualmente en "nueves". El siguiente cuadro muestra cuánto tiempo de inactividad anual tolera cada nivel:

Disponibilidad	Apodo	Inactividad anual aproximada
99 %	Dos nueves	≈ 3,65 días
99,9 %	Tres nueves	≈ 8,76 horas
99,99 %	Cuatro nueves	≈ 52,6 minutos
99,999 %	Cinco nueves	≈ 5,26 minutos

NOTA · RTO y RPO

Dos métricas guían los planes de recuperación ante desastres. El RTO (Recovery Time Objective) es el tiempo máximo aceptable para restaurar el servicio tras una caída. El RPO (Recovery Point Objective) es la cantidad máxima de datos —medida en tiempo— que la organización puede permitirse perder, y determina la frecuencia de los respaldos.

EJEMPLO · Cierre: una arquitectura completa

Una aplicación backend en producción robusta integra todos estos conceptos: el código viaja en contenedores Docker idénticos en cada entorno; se ejecuta en varias instancias sin estado sobre infraestructura en la nube (escalabilidad horizontal); un balanceador reparte el tráfico y retira las instancias caídas (alta disponibilidad); las sesiones y la caché residen en Redis y los archivos en almacenamiento de objetos; y la base de datos se replica con respaldos periódicos para tolerar fallos. Así, el sistema crece con la demanda y sobrevive a la caída de cualquiera de sus piezas.